ОТЧЕТ

АЛМАТИНСКИЙ ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ / ФАКУЛЬТЕТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ

Дисциплина: Проектирование и применение NoSQL-технологий

Вид работы: Самостоятельная работа студента (СРО)

Тема: Разработка NoSQL-подсистемы «GPS-события транспорта» (Вариант 13)

Технология: Apache Cassandra + Python

Выполнил: Абдыкапаров Даниял

Группа: ИС24/22

1. ВВЕДЕНИЕ И ПОСТАНОВКА ЗАДАЧИ

В современных интеллектуальных транспортных системах (ITS) и сервисах управления автопарком (Fleet Management) ключевой задачей является непрерывный сбор, обработка и хранение пространственно-временных данных (телеметрии). Трекеры, установленные на транспортных средствах, отправляют GPS-координаты с высокой частотой (каждые несколько секунд).

Цель мини-проекта:

Проектирование, реализация и тестирование высокопроизводительной NoSQL-подсистемы хранения и обработки GPS-событий транспорта на базе СУБД Apache Cassandra с клиентом на Python.

Задачи:

1. Сформулировать функциональные и нефункциональные требования к системе.
2. Выявить ключевые шаблоны доступа (Access Patterns).

3. Оформить архитектурный документ решения (ADR) с обоснованием выбора Apache Cassandra.
4. Спроектировать CQL-схему данных с учётом бакетирования по дням и порядка кластеризации.
5. Разработать программный модуль (Python) с реализацией не менее 8 типов запросов.
6. Подготовить скрипт генерации синтетических данных (не менее 30 записей) и накрыть функционал интеграционными тестами.

2. ТРЕБОВАНИЯ К СИСТЕМЕ И ПРЕДМЕТНАЯ ОБЛАСТЬ

2.1. Функциональные требования (FR)

1. **FR-1:** Приём и сохранение GPS-телеметрии в реальном времени (широта, долгота, скорость, высота, статус).
2. **FR-2:** Выборка полной истории перемещения конкретного транспортного средства за выбранный день.
3. **FR-3:** Выборка GPS-точек транспортного средства за узкий временной интервал внутри выбранного дня.
4. **FR-4:** Быстрое получение последней известной координаты и статуса автомобиля.
5. **FR-5:** Фильтрация событий превышения скорости за конкретные сутки.
6. **FR-6:** Автоматическое удаление устаревшей телеметрии из хранилища с использованием TTL (Time-To-Live).

2.2. Нефункциональные требования (NFR)

1. **NFR-1 (High Write Throughput):** Оптимизация схемы под интенсивный поток записи (до десятков тысяч записей в секунду) без потери производительности.
2. **NFR-2 (Partition Size Control):** Размер одной партиции не должен превышать рекомендованные 100 МБ благодаря суточному бакетированию (vehicle_id, date).
3. **NFR-3 (High Availability & Scalability):** Архитектура должна обеспечивать горизонтальное масштабирование и линейную деградацию при отказе узлов кластера.

3. ШАБЛОНЫ ДОСТУПА (ACCESS PATTERNS)

Разработка модели данных в NoSQL/Cassandra выполняется строго от запросов (Query-Driven Data Modeling).

№	Наименование паттерна	Элементы запроса (Primary Key)	Приоритет	Ожидаемый результат
AP-1	Запись единичной точки телеметрии	vehicle_id, date, timestamp	Высокий	Успешное сохранение за $O(1)$
AP-2	Пакетная вставка (Batch Insert)	Пачка точек (vehicle_id, date)	Высокий	Сохранение пакета координат
AP-3	Получение трека машины за день	WHERE vehicle_id = ? AND date = ?	Высокий	Отсортированный трек за сутки
AP-4	Выборка по интервалу времени	WHERE vehicle_id = ? AND date = ? AND timestamp >= ? AND timestamp <= ?	Средний	Точки за диапазон часов/минут
AP-5	Получение последней позиции	WHERE vehicle_id = ? AND date = ? LIMIT 1	Высокий	1 свежая запись

AP-6	Фильтрация по превышению скорости	Фильтрация в партиции суточного трека	Низкий	Список нарушений скорости
AP-7	Запись с TTL (автоудаление)	Вставка с ключевым словом USING TTL	Средний	Автоматическое удаление записи
AP-8	Удаление суточного трека	DELETE WHERE vehicle_id = ? AND date = ?	Низкий	Полная очистка партиции

4. СРАВНЕНИЕ АЛЬТЕРНАТИВ И ADR (ARCHITECTURE DECISION RECORD)

4.1. Сравнение альтернативных СУБД

- **Реляционные БД (PostgreSQL / PostGIS):** Отлично подходят для сложных пространственных запросов, но сталкиваются с узким местом по записи при масштабировании на миллионы входящих точек в секунду.
- **Документные БД (MongoDB):** Поддерживают гибкую схему и гео-индексы, однако при сохранении каждой точки отдельным документом требуют частой балансировки шардов, а при группировке в массивы внутри документа сталкиваются с ограничением в 16 МБ.
- **Колоночные distributed БД (Apache Cassandra):** Идеально подходят для Time-Series данных благодаря архитектуре LSM-tree (быстрая запись в Memtable/CommitLog) и встроенному бакетированию через составной первичный ключ.

4.2. Архитектурное решение (ADR-01)

- **Статус:** Принято.
- **Контекст:** Требуется хранилище для непрерывного потока GPS-координат коммерческого автопарка.

- **Решение:** Выбрать **Apache Cassandra** в качестве основного NoSQL-хранилища телеметрии.
- **Последствия:**
    - **Плюсы:** Высокая скорость записи $O(1)$, отсутствие единой точки отказа (Masterless), встроенная поддержка TTL и бакетирования.
    - **Минусы:** Отсутствие JOIN-операций и ограниченные возможности Ad-hoc фильтрации вне структуры Primary Key.

5. МОДЕЛЬ ДАННЫХ И СХЕМА ХРАНЕНИЯ

Для контроля размера партиций и эффективной сортировки выбрана схема суточного бакетирования:

- **Partition Key:** (vehicle_id, date) — отвечает за распределение данных по нодам кластера. Все точки одного автомобиля за один день попадают в одну партицию.
- **Clustering Key:** timestamp — сортирует записи внутри партиции по убыванию времени (CLUSTERING ORDER BY (timestamp DESC)).

5.1. CQL-скрипт инициализации (**scripts/init_db.cql**)

```
CREATE KEYSPACE IF NOT EXISTS transport_tracking
WITH replication = {'class': 'SimpleStrategy', 'replication_factor': 1};

USE transport_tracking;

CREATE TABLE IF NOT EXISTS vehicle_gps_tracks (
    vehicle_id text,
    date text,          -- Формат: 'YYYY-MM-DD'
    timestamp timestamp,
    latitude double,
    longitude double,
    speed double,
    altitude double,
    status text,
    PRIMARY KEY ((vehicle_id, date), timestamp)
) WITH CLUSTERING ORDER BY (timestamp DESC);
```

6. ПРОГРАММНАЯ РЕАЛИЗАЦИЯ И СТРУКТУРА КОДА

Проект реализован на языке Python с использованием драйвера cassandra-driver.

6.1. Архитектура проекта

```
transport-gps-cassandra/
├── src/
│   └── queries.py        # Взаимодействие с Cassandra и 8 запросов
├── data/
│   └── generate_data.py  # Скрипт генерации 30+ синтетических записей
├── scripts/
│   └── init_db.cql       # DDL-скрипт инициализации
├── tests/
│   └── test_tracking.py  # Интеграционные тесты
├── docs/                 # Документация и отчёты
├── asyncore.py           # Заглушка для совместимости с Python 3.12+
├── .env.example
├── requirements.txt
└── README.md
```

6.2. Код основного модуля (src/queries.py)

```
import os
import cassandra.cluster
from cassandra.io.asyncioreactor import AsyncioConnection

cassandra.cluster.DEFAULT_CONNECTION_CLASS = AsyncioConnection

from cassandra.cluster import Cluster
from cassandra.query import BatchStatement
from datetime import datetime, timezone

class GPSHarness:
    def __init__(self, contact_points=['127.0.0.1'], keyspace='transport_tracking'):
        self.cluster = Cluster(contact_points)
        self.session = self.cluster.connect(keyspace)

    def close(self):
        self.cluster.shutdown()

    def insert_point(self, vehicle_id, date, ts, lat, lon, speed, alt, status):
        query = """
            INSERT INTO vehicle_gps_tracks (vehicle_id, date, timestamp, latitude,
longitude, speed, altitude, status)
            VALUES (%s, %s, %s, %s, %s, %s, %s, %s)
        """
        self.session.execute(query, (vehicle_id, date, ts, lat, lon, speed, alt, status))
```

```
    def insert_batch(self, points):
        batch = BatchStatement()
        query = self.session.prepare("""
            INSERT INTO vehicle_gps_tracks (vehicle_id, date, timestamp, latitude,
longitude, speed, altitude, status)
            VALUES (?, ?, ?, ?, ?, ?, ?, ?)
        """)
        for p in points:
            batch.add(query, (p['vehicle_id'], p['date'], p['timestamp'], p['lat'], p['lon'],
p['speed'], p['alt'], p['status']))
        self.session.execute(batch)

    def get_track_by_date(self, vehicle_id, date):
        query = "SELECT * FROM vehicle_gps_tracks WHERE vehicle_id = %s AND
date = %s"
        return list(self.session.execute(query, (vehicle_id, date)))

    def get_track_by_time_range(self, vehicle_id, date, start_ts, end_ts):
        query = """
            SELECT * FROM vehicle_gps_tracks
            WHERE vehicle_id = %s AND date = %s AND timestamp >= %s AND
timestamp <= %s
        """
        return list(self.session.execute(query, (vehicle_id, date, start_ts, end_ts)))

    def get_latest_position(self, vehicle_id, date):
        query = "SELECT * FROM vehicle_gps_tracks WHERE vehicle_id = %s AND
date = %s LIMIT 1"
        rows = list(self.session.execute(query, (vehicle_id, date)))
        return rows[0] if rows else None

    def get_speeding_events(self, vehicle_id, date, min_speed):
        rows = self.get_track_by_date(vehicle_id, date)
        return [row for row in rows if row.speed > min_speed]

    def insert_with_ttl(self, vehicle_id, date, ts, lat, lon, speed, alt, status, ttl_seconds):
        query = """
            INSERT INTO vehicle_gps_tracks (vehicle_id, date, timestamp, latitude,
longitude, speed, altitude, status)
            VALUES (%s, %s, %s, %s, %s, %s, %s, %s) USING TTL %s
        """
        self.session.execute(query, (vehicle_id, date, ts, lat, lon, speed, alt, status,
ttl_seconds))
```

```
    def delete_daily_track(self, vehicle_id, date):
        query = "DELETE FROM vehicle_gps_tracks WHERE vehicle_id = %s AND
date = %s"
        self.session.execute(query, (vehicle_id, date))
```

7. РЕЗУЛЬТАТЫ РАБОТЫ И ДЕМОНСТРАЦИЯ

Для развертывания используется Docker-контейнер Apache Cassandra (cassandra:latest).

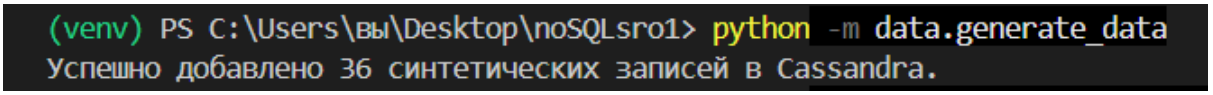
7.1. Запуск и развертывание

База данных запускается одной командой: docker run --name cassandra-node -p 9042:9042 -d cassandra:latest.



7.2. Генерация данных

Скрипт data/generate_data.py генерирует 36 записей для транспортных средств BUS-101, TAXI-202 и TRUCK-303.



7.3. Проверка содержимого через CQLSH

С помощью команды SELECT * FROM vehicle_gps_tracks; подтверждается наличие записей и их верная сортировка по убыванию времени.

```
(venv) PS C:\Users\вы\Desktop\noSQLsro1> docker exec -it cassandra-node cqlsh

ATTENTION: All commands will be saved to history file: /root/.cassandra/cqlsh_history
This may include sensitive information such as passwords.
To disable history, use --disable-history or set 'disabled = true' in the [history] section of cqlshrc.
See https://cassandra.apache.org/doc/latest/tools/cqlsh.html for more information.

Connected to Test Cluster at 127.0.0.1:9042
[cqlsh 6.2.0 | Cassandra 5.0.9 | CQL spec 3.4.7 | Native protocol v5]
Use HELP for help.
cqlsh> USE transport_tracking;
cqlsh:transport_tracking> SELECT * FROM vehicle_gps_tracks;
```

vehicle_id	date	timestamp	altitude	latitude	longitude	speed	status
BUS-101	2026-10-18	2026-10-18 18:20:55.171000+0000	320	51.182	71.462	79.5	IDLE
BUS-101	2026-10-18	2026-10-18 18:16:26.920000+0000	320	51.182	71.462	79.5	IDLE
BUS-101	2026-10-18	2026-10-18 18:10:55.171000+0000	320	51.18	71.46	75	IDLE
BUS-101	2026-10-18	2026-10-18 18:06:26.920000+0000	320	51.18	71.46	75	IDLE
BUS-101	2026-10-18	2026-10-18 18:00:55.171000+0000	320	51.178	71.458	70.5	MOVING
BUS-101	2026-10-18	2026-10-18 17:56:26.920000+0000	320	51.178	71.458	70.5	MOVING
BUS-101	2026-10-18	2026-10-18 17:50:55.171000+0000	320	51.176	71.456	66	MOVING
BUS-101	2026-10-18	2026-10-18 17:46:26.920000+0000	320	51.176	71.456	66	MOVING
BUS-101	2026-10-18	2026-10-18 17:40:55.171000+0000	320	51.174	71.454	61.5	MOVING
BUS-101	2026-10-18	2026-10-18 17:36:26.920000+0000	320	51.174	71.454	61.5	MOVING
BUS-101	2026-10-18	2026-10-18 17:30:55.171000+0000	320	51.172	71.452	57	MOVING
BUS-101	2026-10-18	2026-10-18 17:26:26.920000+0000	320	51.172	71.452	57	MOVING
BUS-101	2026-10-18	2026-10-18 17:20:55.171000+0000	320	51.17	71.45	52.5	MOVING
BUS-101	2026-10-18	2026-10-18 17:16:26.920000+0000	320	51.17	71.45	52.5	MOVING
BUS-101	2026-10-18	2026-10-18 17:10:55.171000+0000	320	51.168	71.448	48	MOVING
BUS-101	2026-10-18	2026-10-18 17:06:26.920000+0000	320	51.168	71.448	48	MOVING
BUS-101	2026-10-18	2026-10-18 17:00:55.171000+0000	320	51.166	71.446	43.5	MOVING
BUS-101	2026-10-18	2026-10-18 16:56:26.920000+0000	320	51.166	71.446	43.5	MOVING
BUS-101	2026-10-18	2026-10-18 16:50:55.171000+0000	320	51.164	71.444	39	MOVING
BUS-101	2026-10-18	2026-10-18 16:46:26.920000+0000	320	51.164	71.444	39	MOVING
BUS-101	2026-10-18	2026-10-18 16:40:55.171000+0000	320	51.162	71.442	34.5	MOVING
BUS-101	2026-10-18	2026-10-18 16:36:26.920000+0000	320	51.162	71.442	34.5	MOVING
BUS-101	2026-10-18	2026-10-18 16:30:55.171000+0000	320	51.16	71.44	30	MOVING
BUS-101	2026-10-18	2026-10-18 16:26:26.920000+0000	320	51.16	71.44	30	MOVING
TRUCK-303	2026-10-18	2026-10-18 18:20:55.171000+0000	320	51.182	71.462	79.5	IDLE
TRUCK-303	2026-10-18	2026-10-18 18:16:26.920000+0000	320	51.182	71.462	79.5	IDLE
TRUCK-303	2026-10-18	2026-10-18 18:10:55.171000+0000	320	51.18	71.46	75	IDLE
TRUCK-303	2026-10-18	2026-10-18 18:06:26.920000+0000	320	51.18	71.46	75	IDLE
TRUCK-303	2026-10-18	2026-10-18 18:00:55.171000+0000	320	51.178	71.458	70.5	MOVING
TRUCK-303	2026-10-18	2026-10-18 17:56:26.920000+0000	320	51.178	71.458	70.5	MOVING
TRUCK-303	2026-10-18	2026-10-18 17:50:55.171000+0000	320	51.176	71.456	66	MOVING
TRUCK-303	2026-10-18	2026-10-18 17:46:26.920000+0000	320	51.176	71.456	66	MOVING
TRUCK-303	2026-10-18	2026-10-18 17:40:55.171000+0000	320	51.174	71.454	61.5	MOVING
TRUCK-303	2026-10-18	2026-10-18 17:36:26.920000+0000	320	51.174	71.454	61.5	MOVING
TRUCK-303	2026-10-18	2026-10-18 17:30:55.171000+0000	320	51.172	71.452	57	MOVING
TRUCK-303	2026-10-18	2026-10-18 17:26:26.920000+0000	320	51.172	71.452	57	MOVING
TRUCK-303	2026-10-18	2026-10-18 17:20:55.171000+0000	320	51.17	71.45	52.5	MOVING
TRUCK-303	2026-10-18	2026-10-18 17:16:26.920000+0000	320	51.17	71.45	52.5	MOVING
TRUCK-303	2026-10-18	2026-10-18 17:10:55.171000+0000	320	51.168	71.448	48	MOVING
TRUCK-303	2026-10-18	2026-10-18 17:06:26.920000+0000	320	51.168	71.448	48	MOVING
TRUCK-303	2026-10-18	2026-10-18 17:00:55.171000+0000	320	51.166	71.446	43.5	MOVING
TRUCK-303	2026-10-18	2026-10-18 16:56:26.920000+0000	320	51.166	71.446	43.5	MOVING
TRUCK-303	2026-10-18	2026-10-18 16:50:55.171000+0000	320	51.164	71.444	39	MOVING
TRUCK-303	2026-10-18	2026-10-18 16:46:26.920000+0000	320	51.164	71.444	39	MOVING
TRUCK-303	2026-10-18	2026-10-18 16:40:55.171000+0000	320	51.162	71.442	34.5	MOVING
TRUCK-303	2026-10-18	2026-10-18 16:36:26.920000+0000	320	51.162	71.442	34.5	MOVING
TRUCK-303	2026-10-18	2026-10-18 16:30:55.171000+0000	320	51.16	71.44	30	MOVING
TRUCK-303	2026-10-18	2026-10-18 16:26:26.920000+0000	320	51.16	71.44	30	MOVING
TAXI-202	2026-10-18	2026-10-18 18:20:55.171000+0000	320	51.182	71.462	79.5	IDLE
TAXI-202	2026-10-18	2026-10-18 18:16:26.920000+0000	320	51.182	71.462	79.5	IDLE
TAXI-202	2026-10-18	2026-10-18 18:10:55.171000+0000	320	51.18	71.46	75	IDLE
TAXI-202	2026-10-18	2026-10-18 18:06:26.920000+0000	320	51.18	71.46	75	IDLE
TAXI-202	2026-10-18	2026-10-18 18:00:55.171000+0000	320	51.178	71.458	70.5	MOVING
TAXI-202	2026-10-18	2026-10-18 17:56:26.920000+0000	320	51.178	71.458	70.5	MOVING

TAXI-202	2026-10-18	2026-10-18 17:50:55.171000+0000	320	51.176	71.456	66	MOVING
TAXI-202	2026-10-18	2026-10-18 17:46:26.920000+0000	320	51.176	71.456	66	MOVING
TAXI-202	2026-10-18	2026-10-18 17:40:55.171000+0000	320	51.174	71.454	61.5	MOVING
TAXI-202	2026-10-18	2026-10-18 17:36:26.920000+0000	320	51.174	71.454	61.5	MOVING
TAXI-202	2026-10-18	2026-10-18 17:30:55.171000+0000	320	51.172	71.452	57	MOVING
TAXI-202	2026-10-18	2026-10-18 17:26:26.920000+0000	320	51.172	71.452	57	MOVING
TAXI-202	2026-10-18	2026-10-18 17:20:55.171000+0000	320	51.17	71.45	52.5	MOVING
TAXI-202	2026-10-18	2026-10-18 17:16:26.920000+0000	320	51.17	71.45	52.5	MOVING
TAXI-202	2026-10-18	2026-10-18 17:10:55.171000+0000	320	51.168	71.448	48	MOVING
TAXI-202	2026-10-18	2026-10-18 17:06:26.920000+0000	320	51.168	71.448	48	MOVING
TAXI-202	2026-10-18	2026-10-18 17:00:55.171000+0000	320	51.166	71.446	43.5	MOVING
TAXI-202	2026-10-18	2026-10-18 16:56:26.920000+0000	320	51.166	71.446	43.5	MOVING
TAXI-202	2026-10-18	2026-10-18 16:50:55.171000+0000	320	51.164	71.444	39	MOVING
TAXI-202	2026-10-18	2026-10-18 16:46:26.920000+0000	320	51.164	71.444	39	MOVING
TAXI-202	2026-10-18	2026-10-18 16:40:55.171000+0000	320	51.162	71.442	34.5	MOVING
TAXI-202	2026-10-18	2026-10-18 16:36:26.920000+0000	320	51.162	71.442	34.5	MOVING
TAXI-202	2026-10-18	2026-10-18 16:30:55.171000+0000	320	51.16	71.44	30	MOVING
TAXI-202	2026-10-18	2026-10-18 16:26:26.920000+0000	320	51.16	71.44	30	MOVING

7.4. Интеграционное тестирование

Тестовый модуль tests/test_tracking.py выполняет проверку функциональности поиска последних координат и фильтрации превышений скорости.

```
(venv) PS C:\Users\вы\Desktop\noSQLsro1> python -m unittest discover -s tests
C:\Users\вы\AppData\Local\Programs\Python\Python314\Lib\asyncio\base_events.py:2047: ResourceWarning: unclosed <socket.socket fd=456, family=2, t
ype=1, proto=0, laddr=('127.0.0.1', 56388), raddr=('127.0.0.1', 9042)>
  handle = None  # Needed to break cycles when an exception occurs.
ResourceWarning: Enable tracemalloc to get the object allocation traceback
C:\Users\вы\AppData\Local\Programs\Python\Python314\Lib\asyncio\base_events.py:2047: ResourceWarning: unclosed <socket.socket fd=520, family=2, t
ype=1, proto=0, laddr=('127.0.0.1', 56389), raddr=('127.0.0.1', 9042)>
  handle = None  # Needed to break cycles when an exception occurs.
ResourceWarning: Enable tracemalloc to get the object allocation traceback
..C:\Users\вы\Desktop\noSQLsro1\src\queries.py:20: ResourceWarning: unclosed <socket.socket fd=460, family=2, type=1, proto=0, laddr=('127.0.0.1'
, 56390), raddr=('127.0.0.1', 9042)>
  self.cluster.shutdown()
ResourceWarning: Enable tracemalloc to get the object allocation traceback

----------------------------------------------------------------------
Ran 2 tests in 1.599s

OK
```

8. ОГРАНИЧЕНИЯ И РЕКОМЕНДАЦИИ

1. **Размер партиций:** При экстремальной частоте отправки данных (например, 100 Гц) суточное бакетирование следует заменить на часовое (vehicle_id, date, hour).
2. **Агрегация:** Для построения сложной аналитики (пробег за месяц, расход топлива) рекомендуется использовать интеграцию с Apache Spark (Cassandra Connector).

9. ВЫВОДЫ

В ходе выполнения самостоятельной работы разработан и протестирован функциональный прототип подсистемы сбора GPS-телеметрии на базе Apache Cassandra. Модель данных с составным первичным ключом ((vehicle_id, date), timestamp) полностью удовлетворяет заданным Access Patterns и гарантирует высокую скорость записи и чтения временных рядов.

10. ССЫЛКИ И ИСТОЧНИКИ

1. Официальная документация Apache Cassandra: https://cassandra.apache.org/doc/latest/
2. DataStax Python Driver Documentation: https://docs.datastax.com/en/developer/python-driver/latest/